AlphaLab v1 — Project Specification

Robustness-Aware Alpha Research Platform — NIFTY 50 Focus

---

1. Project Thesis

Most quant research platforms answer: *is this factor profitable?*
AlphaLab v1 answers: *is this factor still profitable when the data is noisy or incomplete?*

Core research question for v1: **Are profitable alpha factors on NIFTY 50 inherently fragile under noise and missing data?**

Everything in this spec exists to answer that question convincingly, end to end, with nothing left half-built.

---

2. System Architecture

```
Next.js Frontend
      ↓
```

```
FastAPI Backend  ──────────────────► Celery
Worker (Redis broker)
      ↓                                   ↓
  Factor DSL Compiler          Backtest /
Robustness Jobs
      ↓                                   ↓
PostgreSQL (metadata)          DuckDB
(market data, factor values, results)
```

Design principle: flat, single repo. No microservice boundaries, no monorepo package tree. A well-organized single app at this scale is a judgment signal, not a limitation.

---

3. Repository Structure

```
alphalab/
├── api/            # FastAPI app: routes,
schemas, auth
├── worker/         # Celery tasks:
backtest, robustness jobs
├── web/            # Next.js frontend
├── dsl/            # Factor DSL grammar,
parser, compiler
├── engine/         # factor evaluation,
backtesting, robustness scoring
├── data/           # ingestion, cleaning,
DuckDB access layer
├── tests/          # unit + integration
tests
```

```
├── infra/          # docker-compose,
deploy configs
└── docs/           # this spec, research
notes
```

---

4. Technology Stack

Layer	Choice	Why
Frontend	Next.js + TailwindCSS + shadcn/ui	Fast to build, clean defaults
Visualization	Recharts	Equity curves, IC history, robustness heatmap
API	FastAPI	Async support, Python-native, your existing stack from APIDev
Task Queue	Celery + Redis (single queue)	Backtests take 30–60s; real latency problem, not a scaling exercise

Layer	Choice	Why
Metadata DB	PostgreSQL (Neon)	Users, experiments, factor metadata, job status
Analytical DB	DuckDB	Columnar, embedded, fast for rolling-window factor computation over OHLCV
Auth	JWT (PyJWT)	Reuse pattern from APIDev
CI/CD	GitHub Actions → Vercel (web) + Render (api/worker) + Neon (db)	Lint → test → build, proven from ModelMesh

Explicitly excluded from v1: MLflow (Postgres table does the job at this scale), Sentry/structlog (no production traffic yet), microservice monorepo tooling, multi-queue Celery priority tiers.

---

5. Data Layer

Universe

- **NIFTY 50 only** for v1. Extend to NIFTY 100 later without touching pipeline logic.
- Universe must be defined as "**constituents as of date X,**" not a static hardcoded list — avoids survivorship bias in backtests, which is a question you should expect in review.

Source

- **Yahoo Finance** (`.NS` suffix tickers, e.g. `RELIANCE.NS`) for daily OHLCV.
- No dependency on NSE's own feeds for v1 — avoids captcha/rate-limit friction of bhavcopy scraping.

Data Pipeline

```
Yahoo Finance API
      ↓
Ingestion Job (Celery, scheduled daily)
      ↓
Validation Layer
   - missing bar detection
   - suspicious overnight jump detection
(corporate action flag)
   - schema/type checks
      ↓
DuckDB (raw OHLCV table)
      ↓
```

```
Factor Value Computation (on demand, per
experiment)
      ↓
DuckDB (factor_values table)
```

Validation layer detail: Indian equity corporate actions (splits, bonuses, dividends) are noisier on Yahoo than US tickers — flag any overnight price move beyond a threshold (e.g. ±15%) for manual review rather than trusting adjusted close blindly.

---

6. Database Schema

PostgreSQL (metadata)

```
users            (id, email, name,
created_at)
experiments      (id, user_id, name,
description, status, created_at)
                    -- status: PENDING,
RUNNING, FAILED, COMPLETED
factors          (id, experiment_id, name,
formula, created_at)
backtest_results (factor_id, sharpe,
sortino, calmar, max_drawdown, turnover,
ic, rank_ic)
robustness_results (factor_id, noise_score,
missing_data_score, overall_score)
```

DuckDB (analytical warehouse)

```
ohlcv          (ticker, date, open, high,
low, close, volume, adj_close)
universe       (ticker, index_name,
as_of_date)   -- point-in-time constituents
factor_values  (factor_id, ticker, date,
value)
```

---

7. Factor DSL

Why: users should never write arbitrary Python. Safety (no `eval`, no `import os`), leakage detection, and LLM-compatibility (deferred, but designed for) all depend on a closed grammar.

Example expression:

```
Momentum(20) / Volatility(30)
```

Compiler pipeline:

```
DSL string → Parser (AST) → Validator →
Compiler → Executable factor function
```

Validator checks:

- Syntax correctness

- No forward-looking data (leakage) — e.g. rejects any lookahead window
- Computational complexity bound (reject expressions with excessive nested windows)

Supported primitives for v1:

- `Momentum(n)`, `Volatility(n)`, `RollingMean(n)`, `RollingStd(n)`
- Arithmetic operators: `+ - * /`
- No custom Python functions, no I/O, no imports

---

8. Core Engines (v1 scope)

Engine 1: Factor DSL + Compiler

Covered above. Includes validation as part of the compile step (not a separate service).

Engine 2: Backtesting Engine

- **Method:** walk-forward validation (train/test rolls forward through time — no single static split)
- **Outputs:** Sharpe, Sortino, Calmar, Information Coefficient (IC), Rank IC, Max Drawdown, Turnover
- **Execution:** Celery task, since a full NIFTY 50 backtest over several years is the actual latency bottleneck justifying the queue

Engine 3: Robustness Engine (the core differentiator)

Two stress tests only for v1:

1. **Noise Injection** — perturb price/volume by 0.5%, 1%, 2%, remeasure performance
2. **Missing Data** — randomly drop 5%, 10%, 20% of observations, remeasure performance

```
Robustness Score = Average Performance
Under Stress / Original Performance
```

Deferred to v2 (documented, not built): regime-shift testing, cross-market transfer (India ↔ US)

Engine 4: Leaderboard

- Sortable table: factor name, Sharpe, IC, RankIC, Robustness Score
- Backed by a simple Postgres query — no separate service needed

---

9. Explicitly Deferred Features (v2+ roadmap, not v1 deliverables)

Feature	Reason for deferral
Regime Detection (HMM/K-Means)	A thesis-sized ML project on its own — shallow implementation invites unanswerable questions
Cross-Market Transfer (India ↔ US)	Doubles data pipeline scope; only meaningful once India-side robustness thesis is proven
Factor Decay Tracking	Requires months of live data to mean anything — can't be demonstrated from backtests alone
SHAP Explainability	Nice-to-have, not load-bearing for the robustness thesis
LLM-Assisted Factor Generation	Highest risk of looking like a wrapper around a prompt; dilutes the quant-rigor story

Document these in `docs/roadmap.md` as designed-but-deferred — this is itself a talking point, not a gap to hide.

---

10. API Design (FastAPI routes, v1)

```
POST   /experiments                # create
experiment
GET    /experiments/{id}           # get
status + results

POST   /factors                    # submit
DSL formula → validate → compile
GET    /factors/{id}

POST   /factors/{id}/backtest      #
enqueue Celery backtest job
GET    /factors/{id}/backtest      # poll
result

POST   /factors/{id}/robustness    #
enqueue Celery robustness job
GET    /factors/{id}/robustness

GET    /leaderboard                #
sortable factor comparison
```

---

11. Frontend (v1 — two screens only)

1. **Factor Leaderboard** — table of all factors, sortable by Sharpe/IC/RankIC/Robustness
2. **Factor Detail Page** — formula, backtest metrics, equity curve chart, robustness heatmap (noise × missing-data grid)

No regime explorer, no cross-market explorer, no multi-factor comparison screen in v1 — add only if v2 features ship.

---

12. Testing Strategy

- **Unit tests:** DSL parser, validator, metric calculations (Sharpe/IC/RankIC formulas) — target 80% coverage on `dsl/` and `engine/`
- **Integration tests:** factor submission → validation → backtest → result stored in Postgres
- **No E2E/Playwright for v1** — add only once the frontend has enough surface area to justify it

---

13. Build Order (suggested sequence)

1. DuckDB schema + Yahoo Finance ingestion + validation layer
2. Factor DSL grammar + parser + validator + compiler
3. Backtesting engine (walk-forward, core metrics) — run synchronously first, no Celery yet
4. Wrap backtest in a Celery task once you can measure real latency (this is where the Celery

justification becomes concrete, not assumed)

5. Robustness engine (noise + missing data tests)
6. Postgres schema + FastAPI routes wiring it together
7. Leaderboard + Factor Detail frontend
8. CI (lint/test/build) + deploy

---

14. Resume Positioning

Short version:

> Built AlphaLab, a robustness-aware factor research platform that stress-tests NIFTY 50 alpha factors against noise and missing data to distinguish stable signals from overfit ones.

Strong version:

> Designed and built a quant research platform with a custom DSL for safe factor definition (no arbitrary code execution), a walk-forward backtesting engine (Sharpe, IC, RankIC), and a robustness engine quantifying factor stability under noise and missing-data perturbation on NIFTY 50 equities. Introduced async task processing (Celery + Redis) after profiling backtest latency, offloading multi-second jobs from the request path.

Interview talking points:

- Why a DSL instead of raw Python → safety + leakage detection tractability
- Why DuckDB alongside Postgres → columnar analytical vs. relational metadata split
- Why Celery specifically here → measured latency problem, not a checklist item
- What was deferred and why → regime detection / decay / LLM generation each deserve their own thesis; shipping them shallow would have weakened the core robustness claim